

Beyond Autocomplete: A Survey on the Evolution of Human-AI Pair Programming

Nikhil Anand

University of Massachusetts, Amherst

nikhilanand@umass.edu

Abstract

Generative AI tools such as GitHub Copilot and ChatGPT are increasingly integrated into software development workflows, yet their influence on developers' problem solving strategies remains insufficiently understood. This paper presents a literature survey of ten recent studies examining how generative AI affects developer behavior during programming tasks. We adopt a hybrid analytical approach that first defines a taxonomy of AI-assisted interactions and then analyzes their impact across key stages of problem-solving, including problem understanding, solution generation, implementation, and debugging. Our synthesis shows that generative AI shifts developers from exploration-driven problem-solving toward solution selection and refinement, reducing cognitive effort while introducing risks of over-reliance and shallow reasoning. The impact varies across stages: conversational tools primarily support problem understanding, while suggestion-based and generative tools accelerate implementation and debugging. We further identify cross-cutting patterns in trust, prompting behavior, and learning. Finally, we highlight gaps in existing work, including limited longitudinal evidence and insufficient analysis of expertise-dependent effects. These findings provide a structured perspective on how generative AI reshapes developer problem-solving and suggest directions for future research.

CCS Concepts • **Software and its engineering** → **Software development process management**; • **Computing methodologies** → *Multi-agent systems*; Information extraction; Natural language generation

1. Introduction

Generative AI tools powered by large language models (LLMs), such as GitHub Copilot and ChatGPT, are

rapidly becoming integral to software development workflows. These tools provide a wide range of capabilities, including code completion, natural language driven code generation, explanation of programming concepts, and automated debugging assistance. Unlike traditional code recommendation systems, Generative AI enables interactive and iterative engagement through conversational interfaces, allowing developers to specify intent, refine outputs, and explore alternative solutions in real time [1, 13]. Recent empirical studies report widespread adoption of such tools and demonstrate measurable improvements in task completion speed and perceived productivity [2, 10, 11]. As a result, Generative AI is reshaping how developers interact with code and development environments.

Beyond productivity improvements, emerging research has begun to characterize how Generative AI influences developer behavior during programming tasks. Studies have observed changes in code writing practices, debugging strategies, and patterns of interaction with development tools [1, 9]. Developers increasingly engage in prompt based workflows, iteratively refining natural language queries to obtain desired outputs, and often integrate AI-generated code directly into their programs with varying levels of verification [6, 13]. These interactions introduce new dynamics in programming, including shifts in how solutions are explored, how alternatives are evaluated, and how correctness is established. At the same time, prior work highlights variability in usage patterns across developers, tasks, and tool modalities, reflecting differences in expertise, trust, and familiarity with AI-assisted programming environments [6, 9].

To provide a structured understanding of these developments, this paper presents a literature survey of ten recent studies examining developer interactions with Generative AI tools. We adopt a hybrid analytical approach that first defines a taxonomy of AI-assisted programming interactions, capturing distinct modes of engagement such as suggestion-based, conversational, and generative workflows. Building on this taxonomy, we analyze how these interaction types influence different stages of the software development problem-solving process, including problem understanding, solution generation, implementation, and debug-

ging. By synthesizing findings across diverse empirical settings and study designs, this work offers a consolidated view of how Generative AI tools shape developer behavior and interaction patterns in programming tasks.

1.1 Motivation

The dream of an "AI assistant" working alongside the programmer has captured our imagination for several decades now, giving rise to a rich body of work from both the programming languages [5, 8, 12] and the machine learning [Guo et al. 2021; Kalyan et al. 2018; Xu et al. 2020] communities. Thanks to recent breakthroughs in large language models (LLMs) [Li et al. 2022; Vaswani et al. 2017] this dream finally seems within reach. OpenAI's Codex model [Chen et al. 2021], which contains 12 billion model parameters and is trained on 54 million software repositories on GitHub, is able to correctly solve 30–70% of novel Python problems, while DeepMind's AlphaCode [Li et al. 2022] ranked in the top 54.3% among 5000 human programmers on the competitive programming platform Codeforces. With this impressive performance, large code generating models are quickly escaping research labs to power industrial programming assistant tools, such as Github Copilot [Friedman 2021].

Generative AI has emerged as a transformative paradigm across a wide range of domains, enabling systems to produce human-like text, code, and other artifacts with increasing fidelity. Advances in large language models (LLMs) have demonstrated strong capabilities in reasoning, language understanding, and generation, leading to their rapid adoption in real-world applications [3, 4]. These models have shifted the role of AI from purely analytical systems to interactive assistants capable of supporting complex cognitive tasks. As a result, Generative AI is increasingly positioned as a collaborative partner in knowledge work, fundamentally altering how users interact with computational systems and solve problems.

In the context of software development, Generative AI tools such as GitHub Copilot and ChatGPT provide direct support for programming activities, including code generation, explanation, and debugging. Empirical studies have shown that these tools can improve developer productivity, reduce task completion time, and assist in learning unfamiliar concepts [6, 10, 13]. Developers are able to offload routine implementation effort and leverage AI-generated suggestions to accelerate development workflows. At the same time, these tools introduce new interaction patterns, such as prompt-based programming and iterative refinement, which reshape how developers approach coding tasks. This evolution raises important questions about how developers engage with problems, evaluate solutions, and integrate AI assistance into their workflows.

At an industry level, the integration of Generative AI into development environments has significant implications for productivity, collaboration, and software engineering prac-

tices. Organizations are increasingly adopting AI-assisted programming tools to improve efficiency and reduce development costs, while also redefining skill requirements for developers [2, 7]. Despite these advancements, there remains limited understanding of how Generative AI influences the underlying problem-solving strategies of developers across different stages of programming. Existing studies often focus on performance metrics or usability, leaving a gap in understanding the cognitive and behavioral shifts induced by AI assistance. This highlights the need for a structured synthesis of current research to better understand how Generative AI reshapes developer problem-solving and to inform the design of future tools and practices.

1.2 Related Work

References

- [1] S. Barke, M. B. James, and N. Polikarpova. Grounded copilot: How programmers interact with code-generating models. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2023.
- [2] C. Bird et al. Taking flight with copilot: Early insights and opportunities of ai-powered code generation tools. *Queue*, 21(3):35–57, 2023.
- [3] R. Bommasani et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- [4] T. B. Brown et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] K. Ferdowsifard, S. Barke, H. Peleg, S. Lerner, and N. Polikarpova. Loopy: interactive program synthesis with control structures. *Proc. ACM Program. Lang.*, 5(OOPSLA), Oct. 2021. doi: 10.1145/3485530. URL <https://doi.org/10.1145/3485530>.
- [6] M. Kazemitabaar et al. Studying the effect of ai pair programming on developers' tasks. In *CHI Conference on Human Factors in Computing Systems*, 2023.
- [7] McKinsey & Company. The economic potential of generative ai, 2023. Industry report.
- [8] A. Miltner, S. Gulwani, V. Le, A. Leung, A. Radhakrishna, G. Soares, A. Tiwari, and A. Udupa. On the fly synthesis of edit suggestions. *Proc. ACM Program. Lang.*, 3(OOPSLA), Oct. 2019. doi: 10.1145/3360569. URL <https://doi.org/10.1145/3360569>.
- [9] A. Nguyen et al. An empirical study of ai-assisted programming. In *International Conference on Software Engineering (ICSE)*, 2023.
- [10] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590*, 2023.
- [11] N. Perry et al. Github copilot improves developer productivity. *arXiv preprint arXiv:2302.06590*, 2023.
- [12] V. Raychev, M. Vechev, and E. Yahav. Code completion with statistical language models. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14*, page 419–428, New

York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450327848. doi: 10.1145/2594291.2594321. URL <https://doi.org/10.1145/2594291.2594321>.

- [13] P. Vaithilingam, T. Zhang, and E. L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems*, 2022.